Ochrona danych w systemach informatycznych

Laboratorium 3
Szyfrowanie blokowe

Jakub Ciszewski

Semestr 2026Z

Plan i zasady

3 zadania po 1 pkt. Cały kod piszesz samodzielnie.

Dowolny język i biblioteka kryptograficzna.
ECB/CBC: 20 min. Hasło: 30 min. Słaby klucz: 25 min.

ISOD, dokumentacja i własne notatki są dozwolone.
Podczas zajęć wyłącz czaty i podpowiedzi LLM.

Kod na GitHubie. Oddajesz jeden raport TXT/PDF/MD.

AES: blok i klucz

Wariant	Blok	Klucz
AES-128	16 B	16 B
AES-192	16 B	24 B
AES-256	16 B	32 B

Użyj biblioteki do AES, PBKDF2 i PKCS#7.

ECB i CBC

```
ECB: C[i] = AES_K(P[i])
```

```
CBC: C[i] = AES_K(P[i] XOR C[i-1])
```

ECB zachowuje równość powtarzających się bloków.

CBC zależy od poprzedniego bloku kryptogramu.
Dla pierwszego bloku jego rolę pełni IV.

Zadanie 1: cztery takie same bloki

Wejście: 4 × 16 B. Klucz AES-128 podano w B.

Zaszyfruj w ECB i CBC. Wylosuj IV dla CBC.
Wyłącz automatyczne dopełnienie.

Wypisz cztery bloki każdego kryptogramu w hex.
Sprawdź, które bloki są równe.

Odszyfruj oba wyniki i porównaj bajty z wejściem.

Obserwacja: powtórzenia bloków

Własność	ECB	CBC
Długość kryptogramu	64 B	64 B
Cztery równe bloki wejścia	Równe wyniki	Zwykle różne
Powrót do oryginalnych bajtów	Tak	Tak

Równe bloki wejściowe ujawniają wzór w ECB.

PKCS#7: dopełnienie

Dane	Dodane dopełnienie	Kryptogram CBC
0 B	16 bajtów 0x10	16 B
15 B	1 bajt 0x01	16 B
16 B	16 bajtów 0x10	32 B

Dodaj dopełnienie przed szyfrowaniem.
Usuń je po odszyfrowaniu.

Hasło, sól i IV

PBKDF2-HMAC-SHA256: hasło UTF-8 i sól.
32 B klucza, 200000 iteracji.

Sól zmienia klucz wyprowadzony z tego samego hasła.

IV rozpoczyna łańcuch CBC.
Losuj osobno sól i IV, po 16 B.

CBC i poprawne dopełnienie nie dowodzą integralności.

Zadanie 2: hasło i testy

Wyprowadź klucz, zaszyfruj i odszyfruj AES-CBC.
Zastosuj PKCS#7. Zachowaj sól oraz IV.

Napisz testy: puste dane, 16 B, plik wiadomosc.txt.
Sprawdź równość bajtów i długości kryptogramów.

Zaszyfruj wiadomość ponownie z nową solą i IV.
Porównaj kryptogramy.

Raport: wyniki testów oraz rola soli i IV.

16 bajtów klucza, 36 możliwości

Klucz = 16 kopii jednego bajtu ASCII

Alfabet: a–z oraz 0–9. Jeden niezależny wybór.

Entropia bajtów: $H = -\text{suma}(p \times \log_2(p))$.
p to częstość bajtu w próbce; pomijamy p = 0.

Niska entropia może wskazać zwykły tekst.
Dla danych skompresowanych ranking może zawieść.

Zadanie 3: odzyskanie tekstu

Sprawdź 36 kluczy na pierwszych 4096 B pliku.
AES-ECB, bez usuwania dopełnienia z próbki.

Oblicz H i posortuj kandydatów rosnąco.
Wypisz pięciu najlepszych: znak i entropia.

Najlepszym kluczem odszyfruj cały kryptogram.
Usuń PKCS#7 i sprawdź czytelność tekstu.

Raport: top 5, znak klucza, pierwszy wiersz i wniosek.

Oddanie: tylko jeden plik raportu

TXT, PDF albo MD — do jednej strony.

Imię, nazwisko, indeks i język.
Link do GitHuba oraz identyfikator commitu.

Wyniki trzech zadań i krótkie odpowiedzi z B.

Na GitHubie: cały kod i README z uruchomieniem.
Repozytorium musi być dostępne dla prowadzącego.

Pytania przy odbiorze

Dlaczego równe bloki pozostają równe w ECB?

Jaką rolę pełnią sól oraz IV?

Dlaczego powtarzanie znaku daje tylko 36 kluczy?

Za zadanie: 0,50 pkt kod, 0,25 wyniki,
0,25 poprawne wyjaśnienie.

Materiały do przygotowania

ISOD: Laboratorium 3 (Szyfrowanie blokowe).

Dokumentacja wybranej biblioteki:
AES-ECB/CBC, PBKDF2, PKCS#7, losowanie bajtów.

NIST FIPS 197: AES
csrc.nist.gov/pubs/fips/197/final

Przed zajęciami: środowisko, biblioteki i GitHub.
Szczegółowa lista w A-przygotowanie.txt.